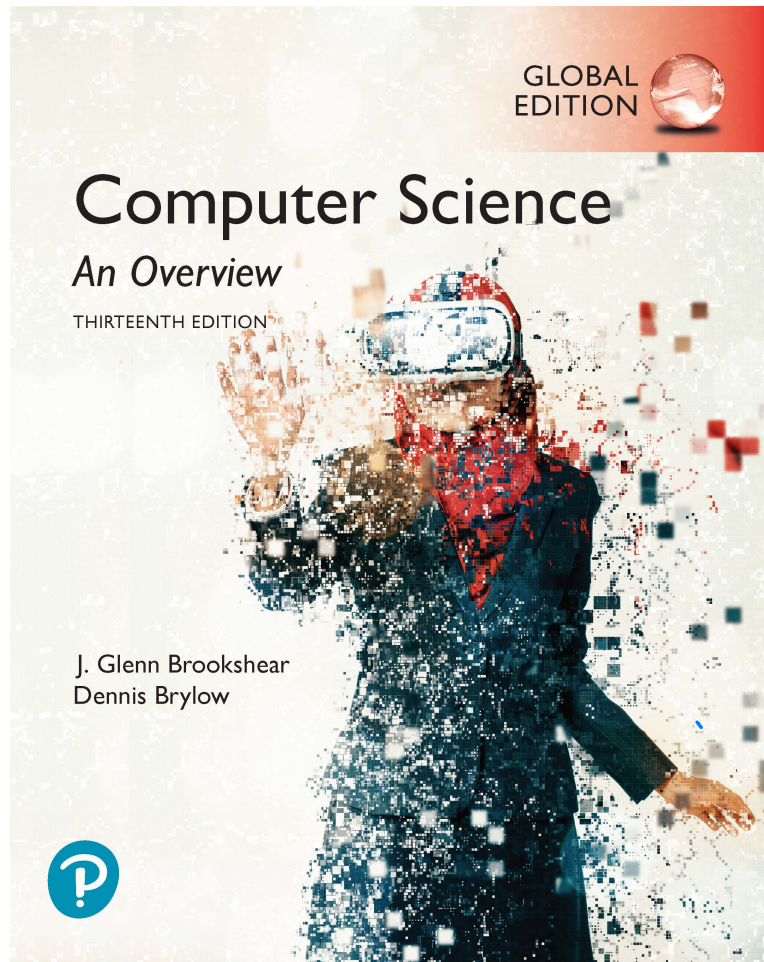


Computer Science An Overview

13th Edition, Global Edition



Chapter 6 Programming Languages

Chapter 6: Programming Languages

- 6.1 Historical Perspective 程式語言的發展
- 6.2 Traditional Programming Concepts
- 6.3 Procedural Units (like function)
- 6.4 Language Implementation 語言的實作
- 6.5 Object Oriented Programming 物件導向概念
- 6.6 Programming Concurrent Activities (活動同步)
- 6.7 Declarative Programming (like SQL Syntax)

6.1 Historical Perspective

* Ver 1, Ver 2 都屬於機器相依的程式語言

- Early Generations
 - Machine Language (e.g. Vole) 機器語言 (0,1) 第一代程式語言
 - Assembly Language 組台語言 第二代程式語言
- Machine Independent Language 非機器相依程式語言 (C/C++, Java)
- Beyond – more powerful abstractions

Second-generation: Assembly language

- A mnemonic system for representing machine instructions 表示機器指令的 助記系統
- Mnemonic names for op-codes
- Program **variables** or **identifiers**: Descriptive names for memory locations, chosen by the programmer

Assembly Language Characteristics

- One-to-one correspondence between machine instructions and assembly instructions
 - Programmer must think like the machine
- Inherently machine-dependent
- Converted to machine language by a program called an **assembler**

└ 將組合語言轉譯成機器語言 — 組譯器

* 有了變數的概念

Program Example

Machine language

*前幾張學過的 op-code

156C

166D

5056

30CE

C000

Assembly language

加上了變數的概念!

LD R5, Price

LD R6, ShipCharge

ADDI R0, R5 R6

ST R0, TotalCost

HLT

Third Generation Language 高階程式語言

- Uses high-level primitives 使用較貼近人類原始語言的表達方式!
- Similar to our pseudocode in Chapter 5
- Machine independent (mostly) 非機器相依的 Programming Language
- Examples: FORTRAN, COBOL
- Each primitive corresponds to a sequence of machine language instructions
- **Converted to machine language by a program called a compiler**

將 High-level 程式語言轉成機器語言的工具 — 編譯器

Figure 6.1 Generations of programming languages

人類的思考邏輯必須順應機器

Problems solved in an environment in which the human must conform to the machine's characteristics.

機器去順應人類的思考邏輯、語言特性

Problems solved in an environment in which the machine conforms to the human's characteristics.

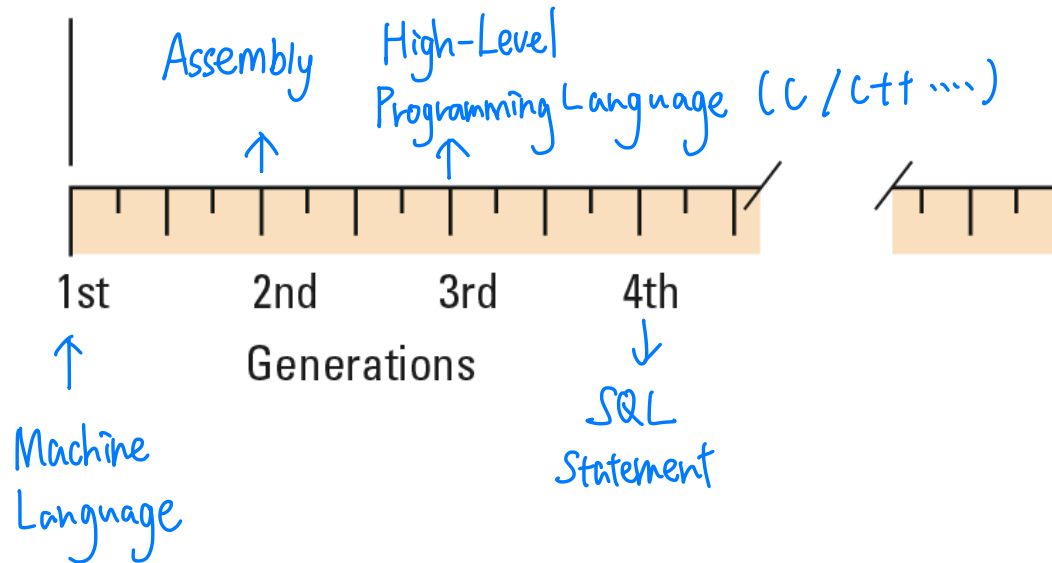


Figure 6.2 The evolution of programming paradigms

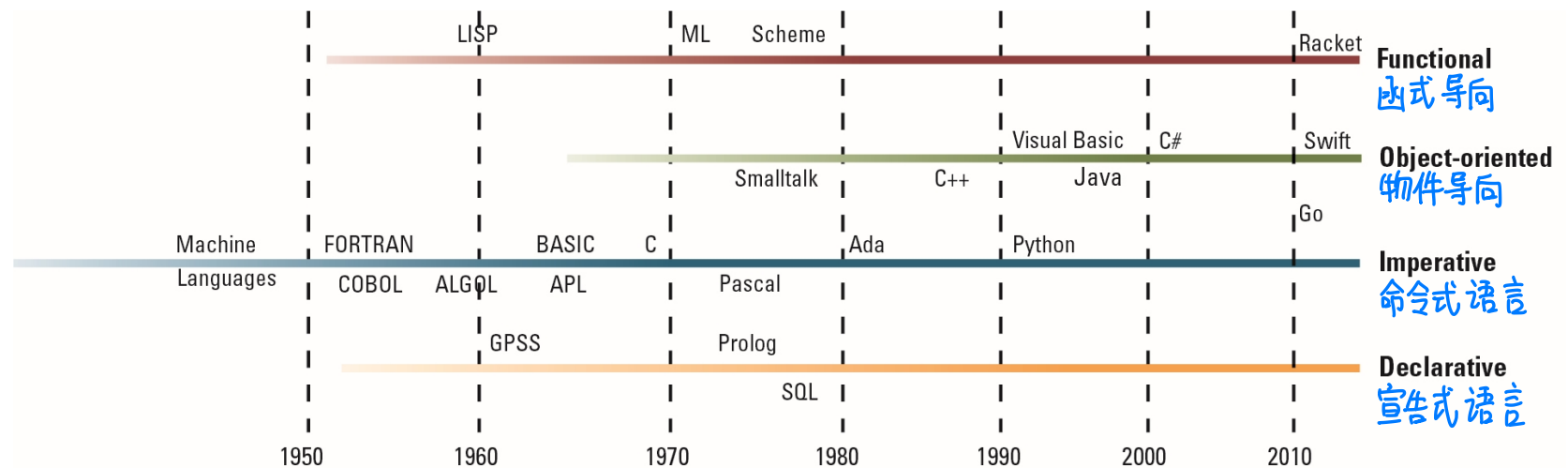
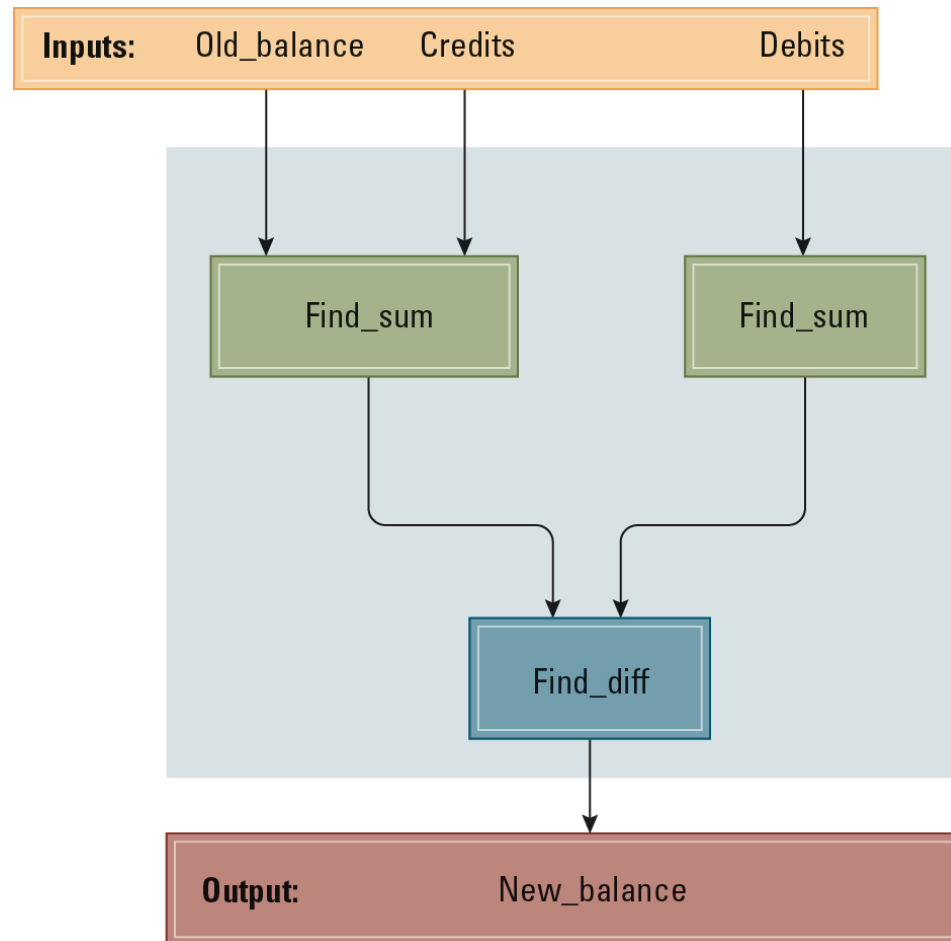


Figure 6.3 A function for checkbook balancing constructed from simpler functions



6.2 Traditional Programming Concepts

高階程式語言可能包含許多抽象概念

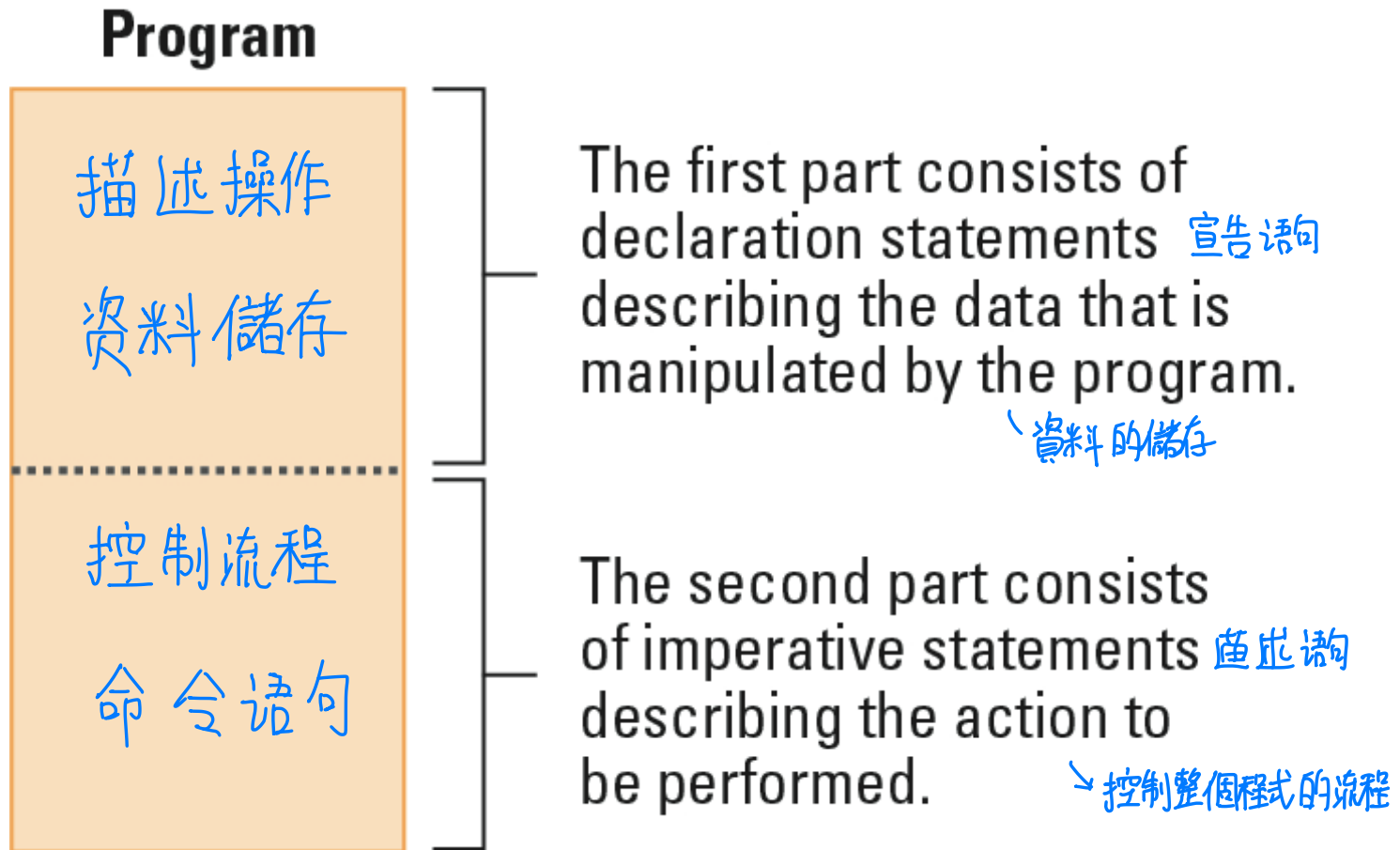
- High-level languages (C, C++, Java, C#, FORTRAN) include many kinds of abstractions
 - Simple: constants, literals, variables
常數 定數 變數
 - Complex: statements, expressions, control
 - Esoteric: procedures, modules, libraries
副程式 模組 函式庫

控制语句 Control Statements:

goto, if-else, while, switch, for-loop

Figure 6.4 The composition of a typical imperative program or program unit

程式的組成



Data Types

- Integer: Whole numbers
- Real (float): Numbers with fractions → 还可以区分为单精度 & 双精度
(float) (double)
- Character: Symbols
- Boolean: True/false

Variables and Data types

```
float Length, Width;
```

```
int Price, Total, Tax;
```

```
char Symbol;
```

```
int WeightLimit = 100;
```

Data Structure 資料結構

- Conceptual shape or arrangement of data
- A common data structure is the **array**

In C

```
int Scores[2][9];
```

In FORTRAN

```
INTEGER Scores(2,9)
```

Figure 6.5 A two-dimensional array with two rows and nine columns

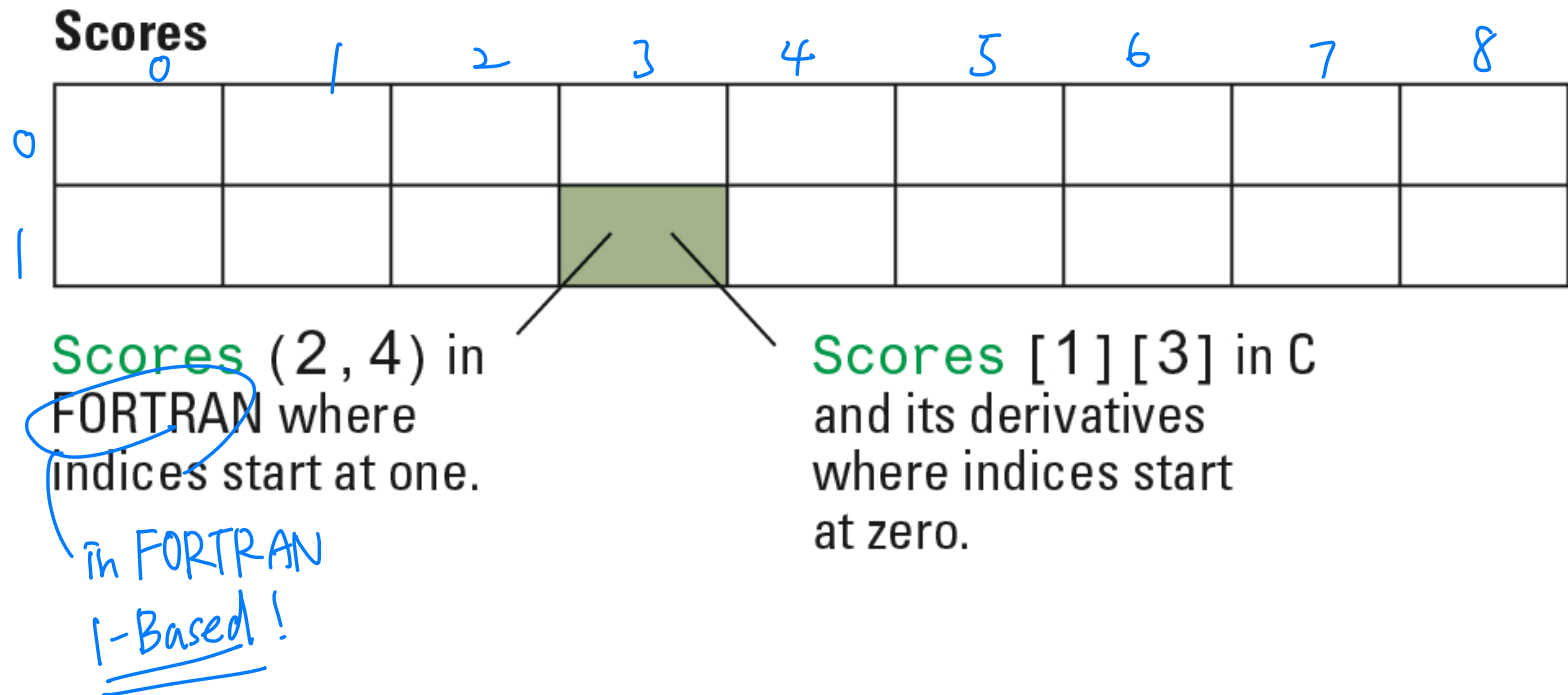
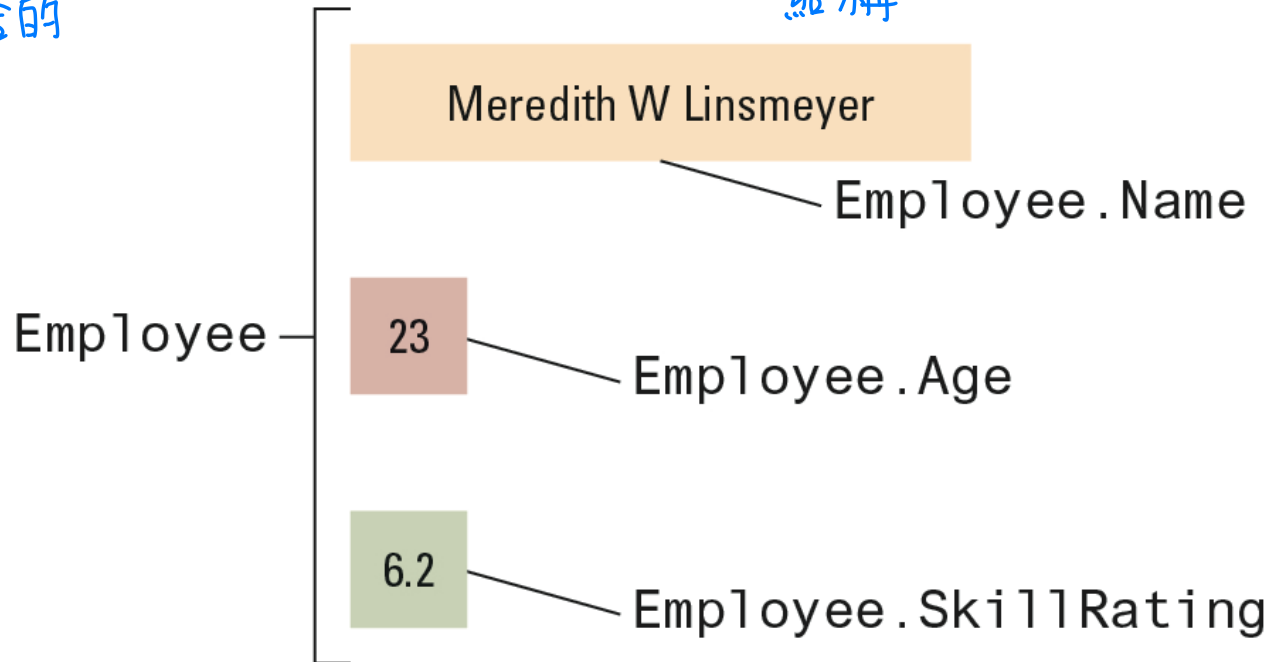


Figure 6.6 The conceptual structure of the aggregate type Employee

adj. 聚合的

結構



```
struct { char Name[25];  
        int Age;  
        float SkillRating;  
    } Employee;
```

Assignment Statements

運算、賦值 语句

- In C, C++, C#, Java

$Z = X + y;$

- In Ada

$Z := X + y;$

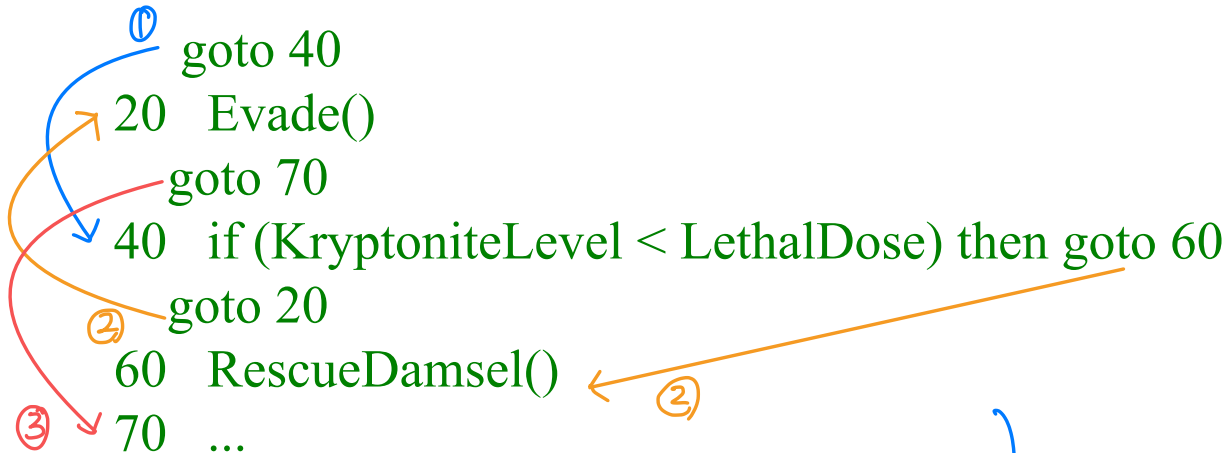
- In APL (A Programming Language)

$Z \leftarrow X + y$

Control Statements

控制语句

- Go to statement



- As a single statement

```
if (KryptoniteLevel < LethalDose):  
    RescueDamsel()  
else:  
    Evade()
```

← 改寫成 if 結構

(*可能會考!)

Control Statements (continued)

- If in Python

```
if (condition):  
    statementA  
else:  
    statementB
```

- In C, C++, C#, and Java

```
if (condition) statementA; else statementB;
```

- In Ada

```
IF condition THEN  
    statementA;  
ELSE  
    statementB;  
END IF;
```

Control Statements (continued)

- While in Python

```
while (condition):  
    body
```

- In C, C++, C#, and Java

```
while (condition)  
{ body }
```

- In Ada

```
WHILE condition LOOP  
    body  
END LOOP;
```

Control Statements (continued)

P.342

*類似書籤

- Switch statement in C, C++, C#, and Java

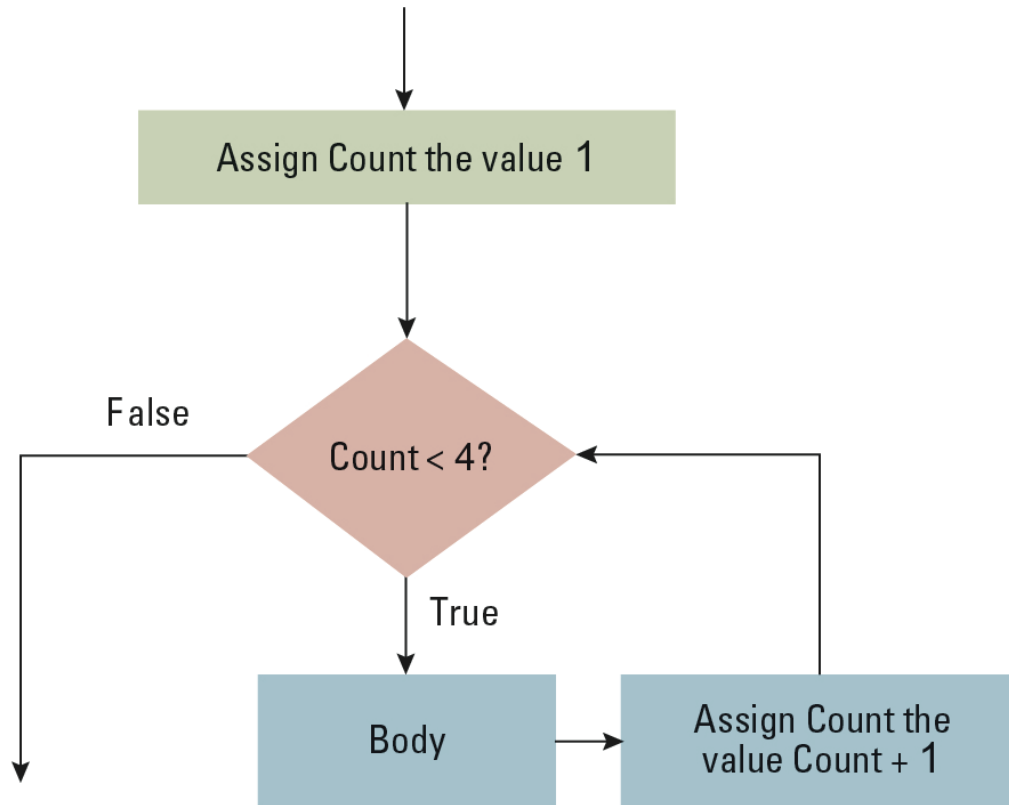
```
switch (variable) {  
    case 'A': statementA; break;  
    case 'B': statementB; break;  
    case 'C': statementC; break;  
    default: statementD; }
```

若沒有break, 且A成立, 則B、C、default都會被執行!

- In Ada *Ada不用break!

```
CASE variable IS  
    WHEN 'A'=> statementA;  
    WHEN 'B'=> statementB;  
    WHEN 'C'=> statementC;  
    WHEN OTHERS=> statementD;  
END CASE;
```

Figure 6.7 The **for loop structure** and its representation in C++, C#, and Java



```
for (int Count = 1; Count < 4; Count++)  
    body;
```

Comments 註解

- Explanatory statements within a program
- Helpful when a human reads a program
- Ignored by the compiler

`/* This is a comment in C/C++/Java. */`

`// This is a comment in C/C++/Java.`

6.3 Procedural Units

程式單元

- Many terms for this concept:

- Subprogram, subroutine, procedure, method, function
子程式 副程式 程序 方法
→ 这些都是「程式單元」(Procedural Unit)
函式

- Unit begins with the function's **header**

標頭

⇒ 每個程式單元

like void, int, bool

皆以標頭開始!

- Local versus Global Variables

- Formal versus Actual Parameters

定義時
形式參數

呼叫時(傳參)
實質參數

- Passing parameters by value versus reference

值 v.s. 址
(copy) (same)

fruitful function: 有回傳值的函式

Passing Parameters by Values 傳值: copy 內容至 function 內的區域 variables, 對新 variables 操作

Passing Parameters by reference 傳址: 將 Variables 記憶體位置傳給 function, 對原本 Variables 直接操作

Figure 6.8 The flow of control involving a function

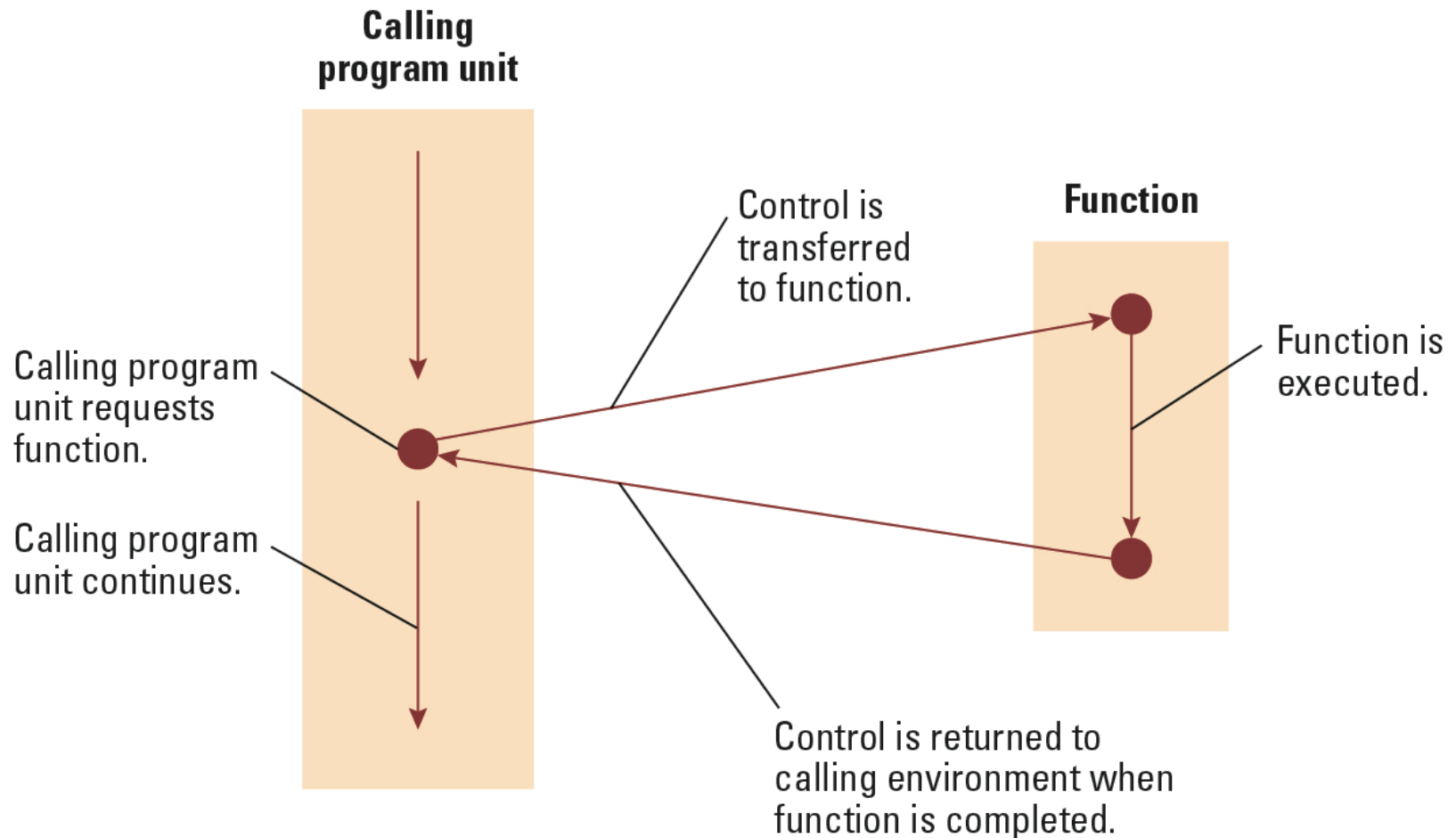


Figure 6.9 The function ProjectPopulation written in the programming language C

Starting the header with the term “void” is the way that a C programmer specifies that the program unit returns no value. We will learn about return values shortly.

Header
↓ 回傳值的形態

The **formal parameter** list. Note that C, as with many programming languages, requires that the data type of each parameter be specified.

```
void ProjectPopulation (float GrowthRate)
{
    int Year; // Local variables // This declares a local variable named Year.
    Population[0] = 100.0; // global variables
    for (Year = 0; Year <= 10; Year++)
        Population[Year+1] = Population[Year] + (Population[Year] * GrowthRate);
}
```

These statements describe how the populations are to be computed and stored in the global array named Population.

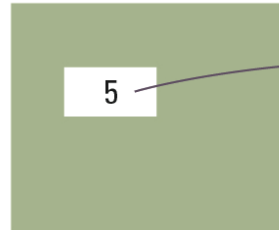
Figure 6.10

Executing the function Demo and passing parameters by value

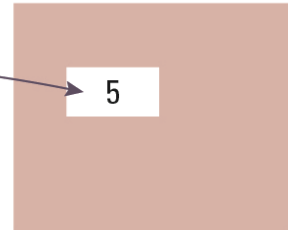
傳值: copy 內容至function內的
新 variables

- a. When the function is called, a copy of the data is given to the function

Calling environment



Function's environment

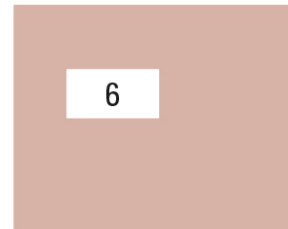


- b. and the function manipulates its copy.

Calling environment



Function's environment



- c. Thus, when the function has terminated, the calling environment has not been changed.

Calling environment

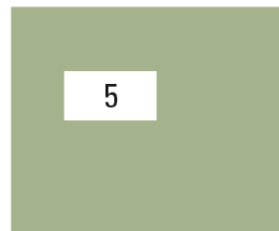


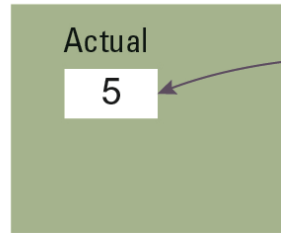
Figure 6.11

Executing the function Demo and passing parameters by reference

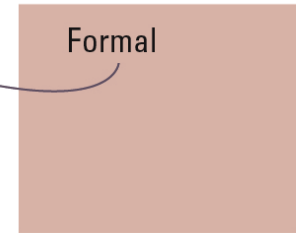
傳址：將 variables 的
記憶體位置傳給
function，對原本的
variables 直接操作。

- a. When the function is called, the formal parameter becomes a reference to the actual parameter.

Calling environment

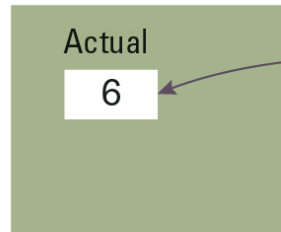


Function's environment

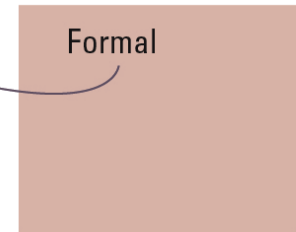


- b. Thus, changes directed by the function are made to the actual parameter

Calling environment



Function's environment



- c. and are, therefore, preserved after the function has terminated.

Calling environment

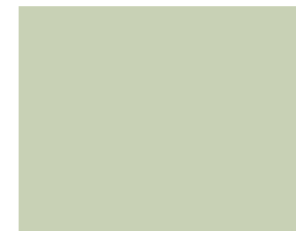
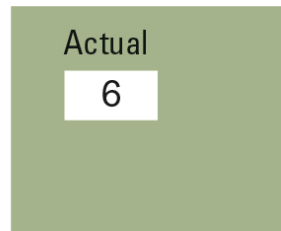


Figure 6.12 The fruitful function CylinderVolume written in the programming language C

有回傳值的函數

The function header begins with the type of the data that will be returned.

```
float CylinderVolume (float Radius, float Height)
```

```
{float Volume;
```

Declare a
local variable
named Volume.

```
Volume = 3.14 * Radius * Radius * Height;
```

```
return Volume;
```

Compute the volume of
the cylinder.

```
}
```

Terminate the function and
return the value of the
variable Volume.

6.4 Language Implementation

語言的實作
(編譯過程的實作)

- The process of converting a program written in a high-level language into a machine-executable form.
- The ^{詞法分析器} Lexical Analyzer recognizes which strings of symbols represent a single entity, or token. → 將Statement拆解成單獨實體、函數、token...
- ^{分群} ^{重組} The ^{語法分析器} Parser groups tokens into statements, using syntax diagrams to make parse trees. 目標: 建構 Parse Trees
- The Code Generator constructs machine-language instructions to implement the statements.
根據 Parse Trees 產生 Machine instruction

Figure 6.13 The translation process

P.355

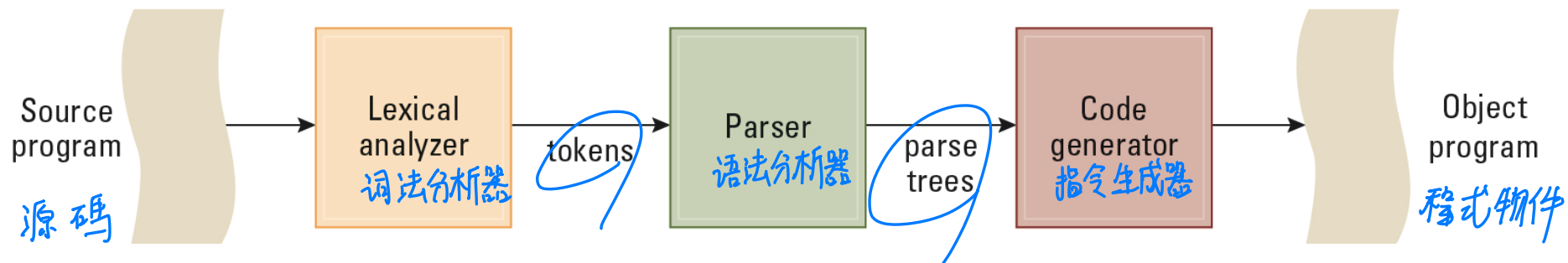
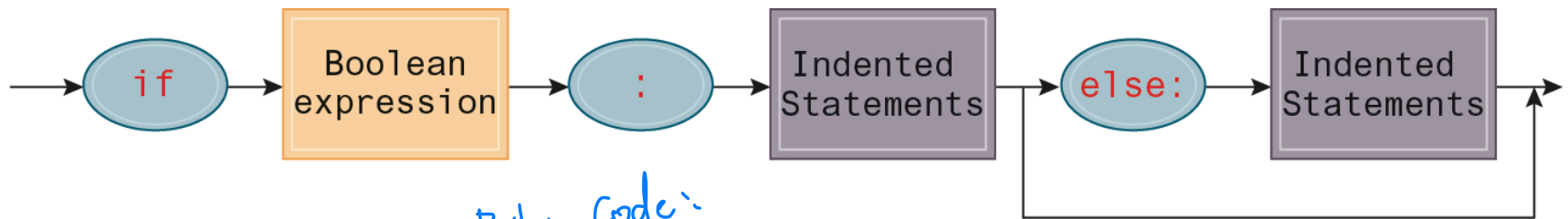


Figure 6.14 A syntax diagram 语法图 of Python's if-then-else statement

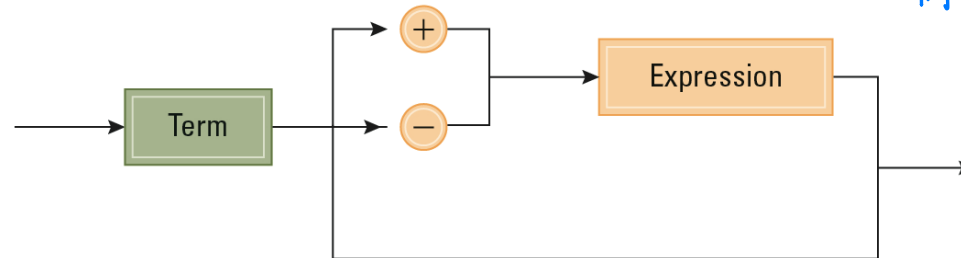


Convert to Python Code:-
if Boolean_expression:
 Indented_Statements
else:
 Indented_Statements

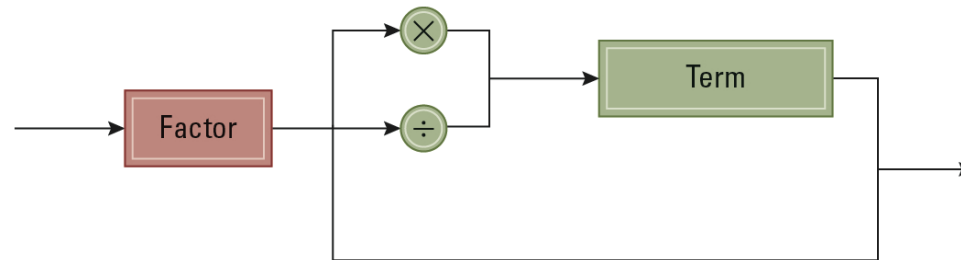
Figure 6.15 Syntax diagrams describing the structure of a simple algebraic expression

簡單代數 運算語法圖

Expression



Term



Factor

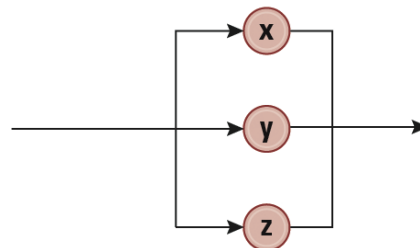


Figure 6.16 The parse tree for the string $x + y * z$ based on the syntax diagrams in Figure 6.17

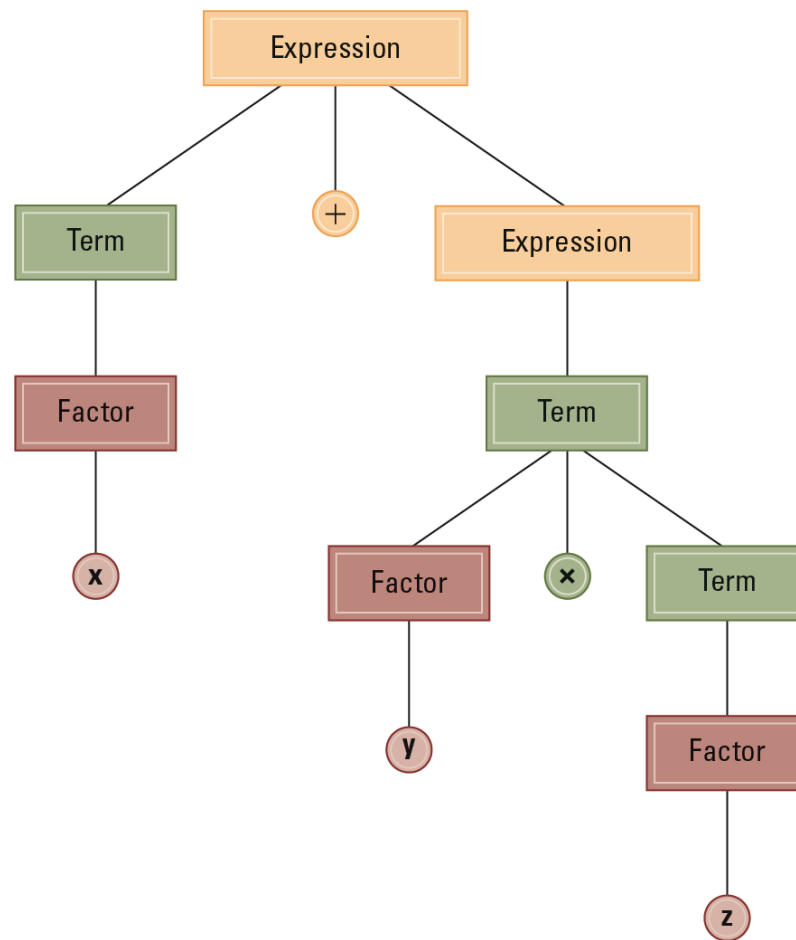


Figure 6.17 Two distinct parse trees for the statement if B1 then if B2 then S1 else S2

不同的縮排
可能產生不同的 Parse Trees

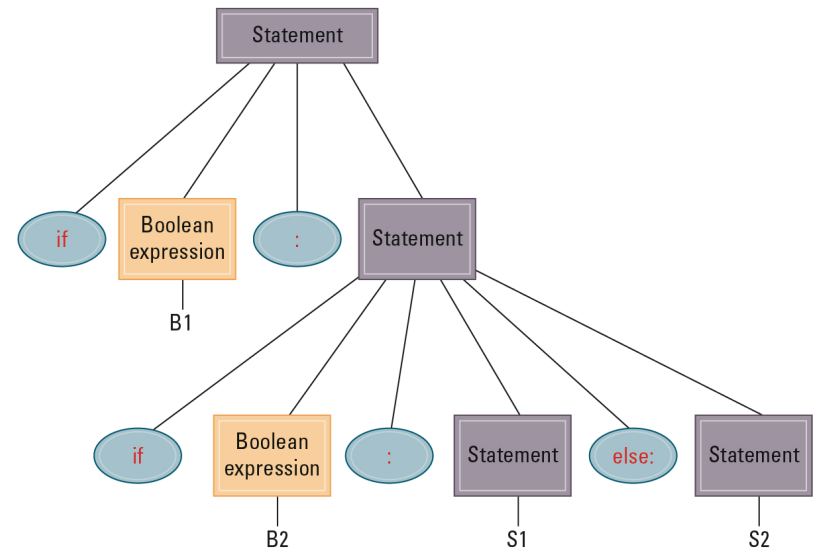
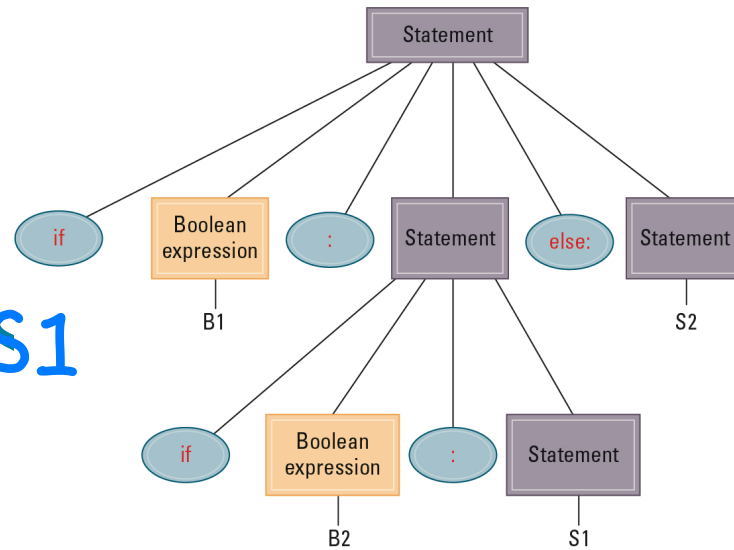
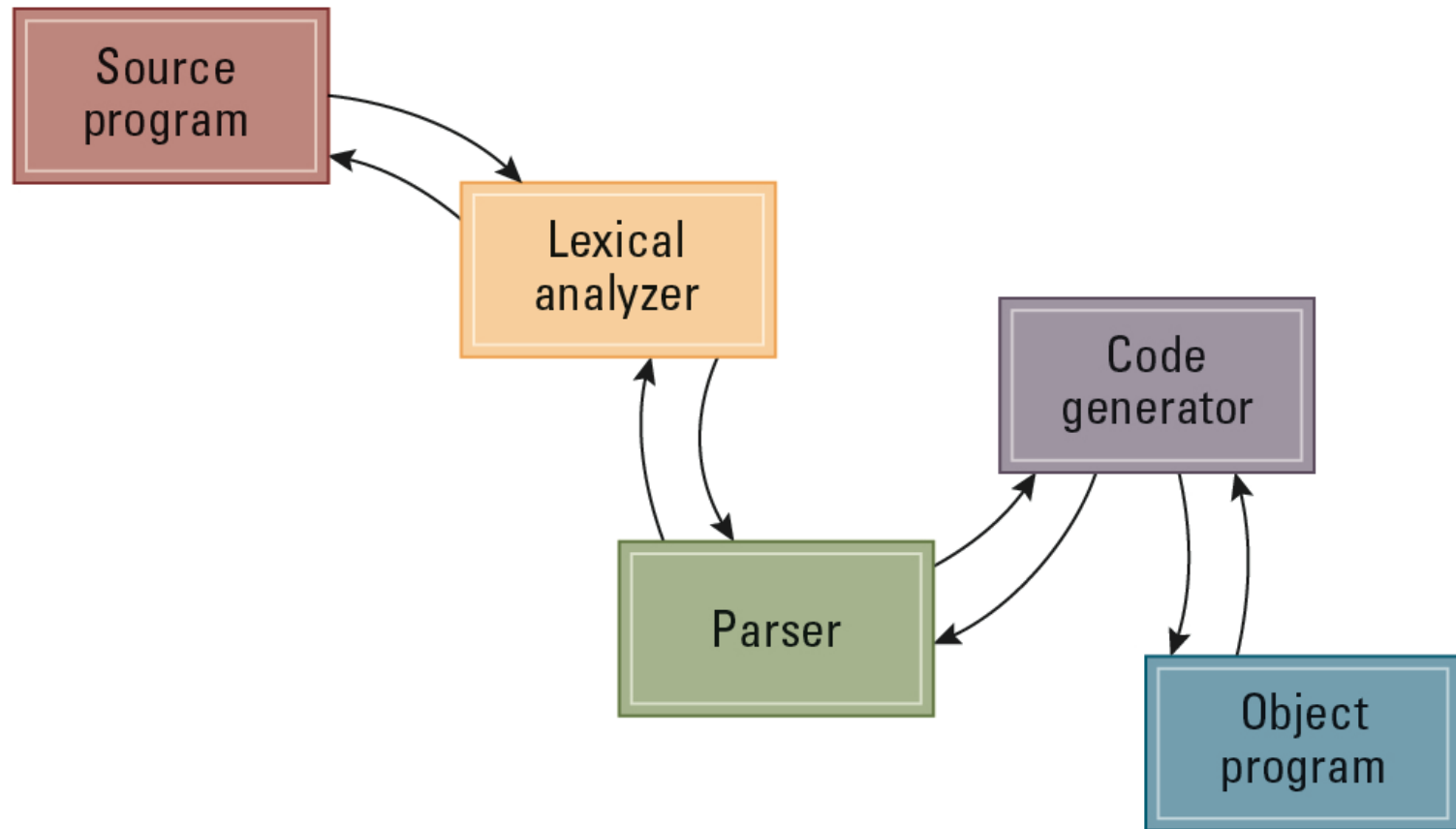


Figure 6.18 An object-oriented approach to the translation process



物件导向程式 (OOP)

课. P364

6.5 Object-Oriented Programming

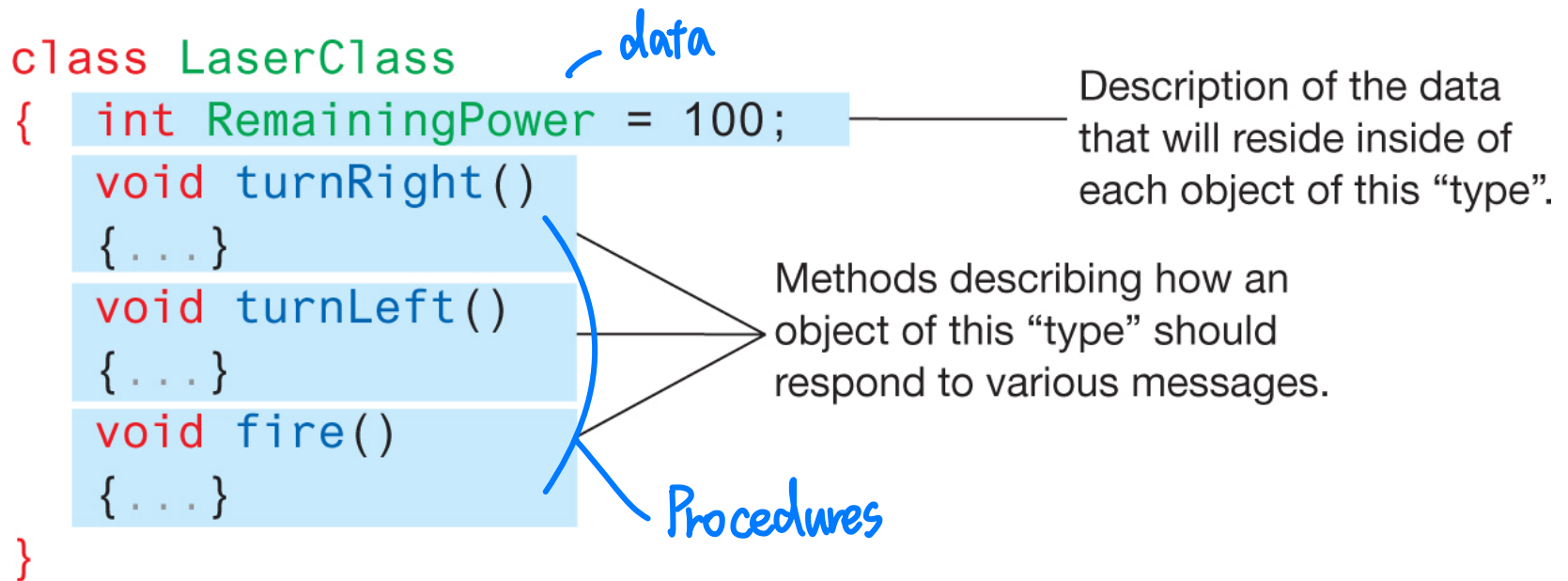
- **Object:** Active program unit containing both data and procedures → 物件: 包含资料和一些函数(子程式); 必需透过 Class 来定义、产生。
- **Class:** A template from which objects are constructed \ 生成物件 (Constructed Object) 的模板, 有了 Class 才能够产生 Object。

An object is called an instance of the class.

\ 物件 是 類別的 实体化!

一個物件(object) 被稱為類別实例

Figure 6.19 The structure of a class describing a laser weapon in a computer game



Components of an Object

- **Instance Variable:** Variable within an object
 - Holds information within the object 实例變數 (物件當中的變數)
- **Method:** Procedure within an object (物件當中的 Method/func)
 - Describes the actions that the object can perform
- **Constructor:** Special method used to initialize a new object when it is first constructed 建構子: 新物件首次被建立時用於初始化的特殊物件

記得看課本!
(~ P370)

Figure 6.21 A class with a constructor

```
class LaserClass  
{ int RemainingPower;
```

```
LaserClass(InitialPower)  
{ RemainingPower = InitialPower;  
}
```

Constructor assigns a value to RemainingPower when an object is created.

like 函數傳值

```
void turnRight()  
{ ... }
```

```
void turnLeft()  
{ ... }
```

```
void fire()  
{ ... }
```

```
}
```

P.366 實體化物件

LaserClass Laser1 = new LaserClass();

Object Integrity

封裝

- **Encapsulation:** A way of restricting access to the internal components of an object
 - Private
 - Public

限制物件內 Methods 的存取行為

Figure 6.22 Our LaserClass definition using encapsulation as it would appear in a Java or C# program

Components in the class are designated public or private depending on whether they should be accessible from other program units.

```
class LaserClass
{private int RemainingPower;
public LaserClass (InitialPower)
{RemainingPower = InitialPower;
}
public void turnRight ( )
{...}
public void turnLeft ( )
{...}
public void fire ( )
{...}
}
```

Additional Object-oriented Concepts

繼承

- **Inheritance**: Allows new classes to be defined in terms of previously defined classes
- **Polymorphism**: Allows method calls to be interpreted by the object that receives the call

多形化

P.369

、允許由呼叫的物件解釋呼叫的函數(methods)

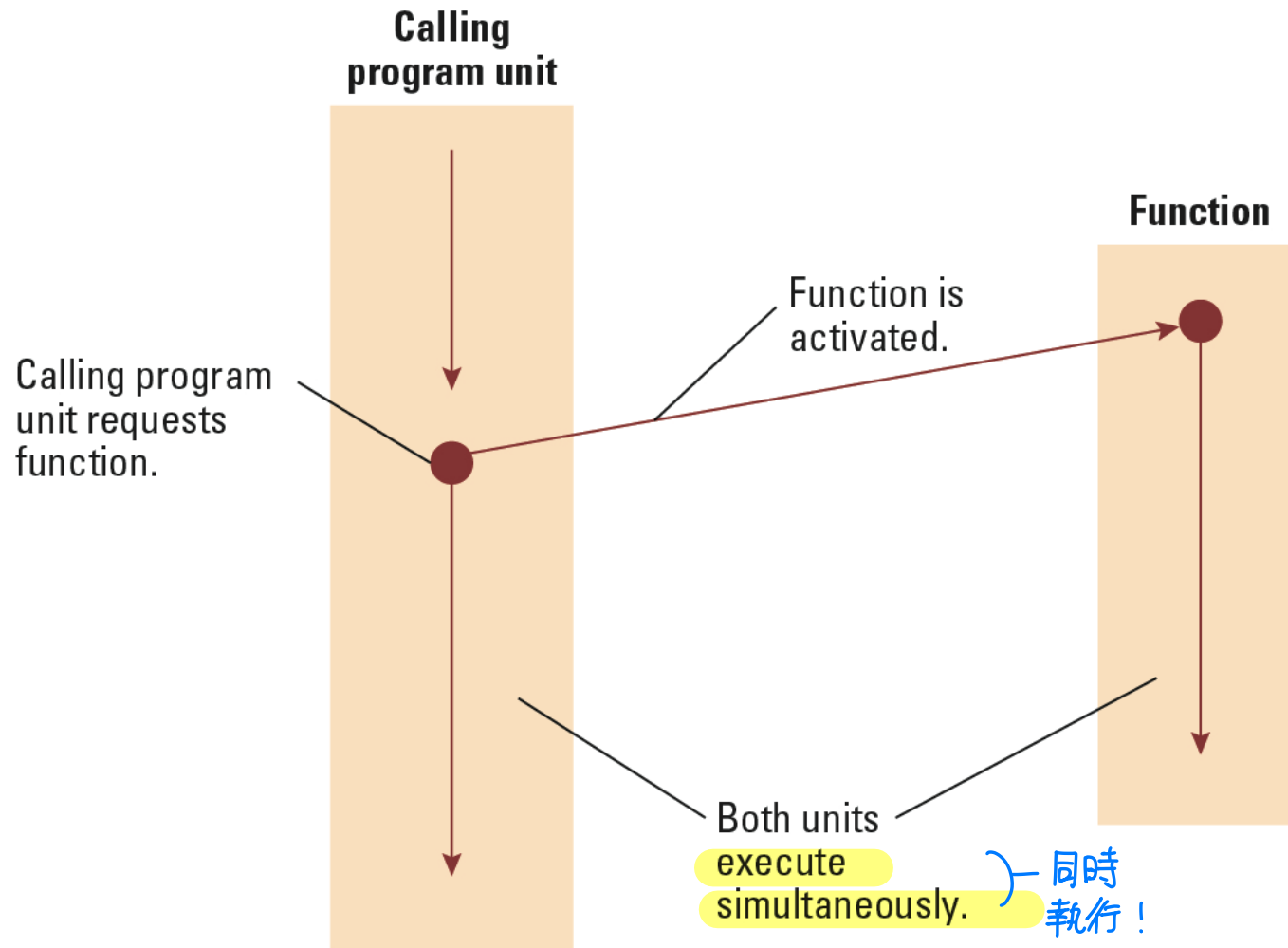
6.6 Programming Concurrent Activities

平行運算

- **Parallel** (or **concurrent**) processing: **simultaneous** execution of multiple processes
 - True concurrent processing requires multiple CPUs
 - Can be simulated using time-sharing with a single CPU

Hint! 分時系統 可以達到類似效果

Figure 6.23 Spawning threads



Controlling Access to Data

互斥鎖：確保資料一次只被一個 Process 存取

- **Mutual Exclusion:** A method for ensuring that data can be accessed by only one process at a time

- **Monitor:** A data item augmented with the ability to control access to itself

監控：增強一個項目控制對其自身存取的能力

強制轉換 / 強迫 (Coercion)：課 P. 361

如果 x 的類型是整數， y 的類型是浮點數，則會出現一種特殊情況。在這種情況下，加法的概念仍然適用，但這兩個值並未以兼容的形式編碼。在這種情況下，語法分析器可能會選擇讓程式碼生成器構建指令，將其中一個值轉換為另一種型別，然後進行加法運算。這種型別之間的隱式轉換稱為強制轉換 (coercion)。

6.7 Declarative Programming

宣告式

將多個語句組合起來，產生新的語句

- **解析 Resolution:** Combining two or more statements to produce a new statement (that is a logical consequence of the originals).
 - Example: $(P \text{ OR } Q) \text{ AND } (R \text{ OR } \neg Q)$ resolves to $(P \text{ OR } R)$
 - **結果 Resolvent:** A new statement deduced by resolution
 - **句形 Clause form:** A statement whose elementary components are connected by the Boolean operation OR
- **統一 Unification:** Assigning a value to a variable so that two statements become “compatible.”
為變數賦值，s.t. 兩段語句“兼容”

Figure 6.24 Resolving the statements $(P \text{ OR } Q)$ and $(R \text{ OR } \neg Q)$ to produce $(P \text{ OR } R)$

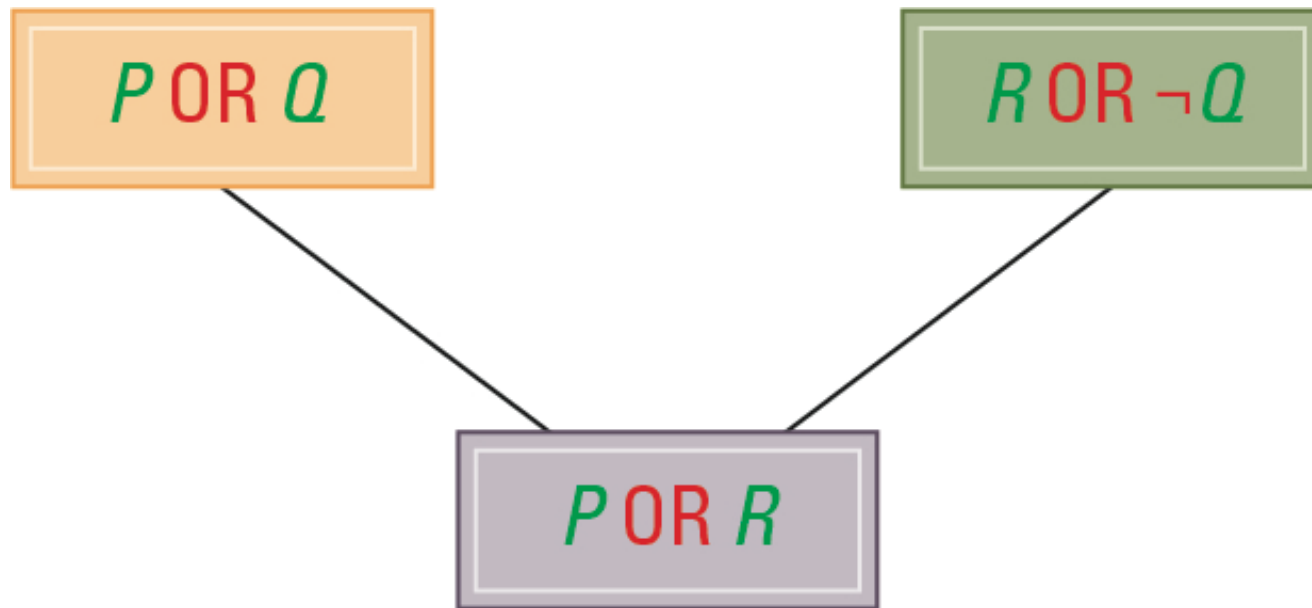
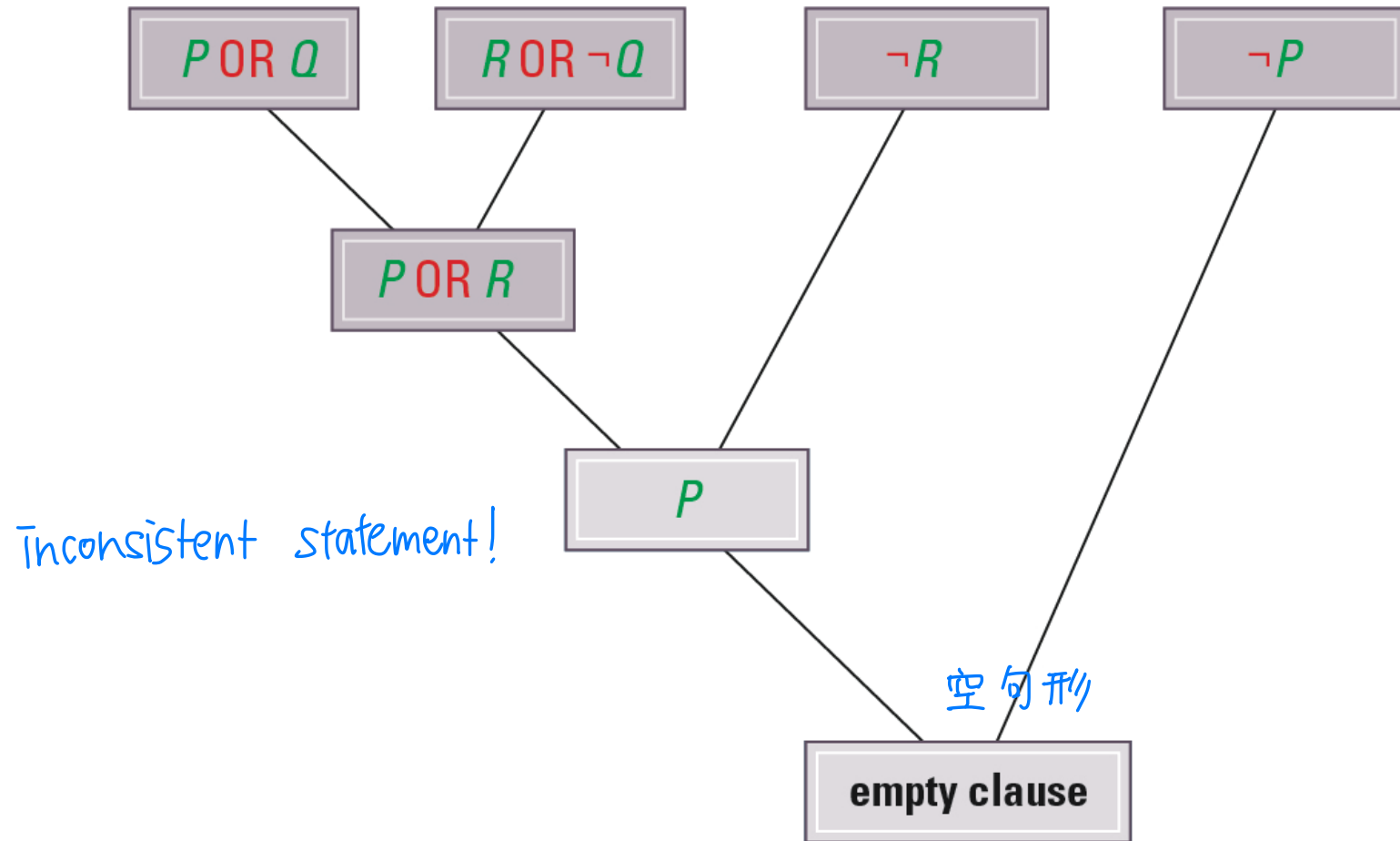


Figure 6.25 Resolving the statements $(P \text{ OR } Q)$, $(R \text{ OR } \neg Q)$, $\neg R$, and $\neg P$



Prolog

事实：一个在 Prolog 中建立“情况”的语句

- **Fact:** A Prolog statement establishing a fact

- Consists of a single predicate → 单一谓词组成

- Form: *predicateName*(arguments).
谓词名称

- Example: `parent(bill, mary) .`

規則

- **Rule:** A Prolog statement establishing a general rule

- Form: *conclusion* :- *premise*. 建立一个一般化的規則

- :- means “if”

- Example: `wise(X) :- old(X) .`

- Example: `faster(X,Y) :- faster(Y,Z) ,
faster(X,Z) .`